Sisteme de operare
Laborator 11
Demoni si servere Web

1. Creati un nou subdirector *lab11/* in structura de directoare a laboratorului creata anterior (*SO/laborator*) si subdirectoarele aferente *doc src* si *bin*. Nu uitati sa actualizati variabila de mediu PATH pentru a include directorul *SO/laborator/lab11/bin*.

2. Scrieti un program **mmap-cp.c** care copiaza un fisier sursa intr-un fisier destinatie folosind apelul sistem *mmap*. Numele fisierelor sunt furnizate in linie de comanda ca mai jos:

$ mmap-cp <fisier-sursa> <fisier-destinatie>

si programul le mapeaza in memorie cu ajutorul *mmap* inainte de a face copierea datelor.

Dupa ce ati testat corectitudinea programului, folositi un fisier de dimensiune relativ mare (eg, cca 100MB) drept fisier sursa si comparati performanta **mmap-cp** cu cea a programului **mycp** din laboratorul 1 folosind comanda *time* in felul urmator:

```
$ time ./mmap-cp fisier-100MB copie-mmap
$ time ./mycp < fisier-100MB > copie-mycp
```

Ce anume observati in timpii afisati de comanda *time*? De unde credeti ca vin diferentele dintre timpii petrecuti in user- si respectiv system-space de catre cele doua programe?

3. Scrieti o functie *daemon_init* care transforma un server intr-un demon. Cititi cu atentie slide-urile de laborator si implementati fiecare din pasii necesari. Semnatura functiei este urmatoarea:

*int daemon_init(const char *pname, int facility);*

Functia intoarce valoarea -1 daca esueaza (i.e., nu poate transforma serverul in demon) sau 0 in caz de succes. Primul parametru este numele serverului iar al doilea este valoarea de facilitate folosita de *syslog*.

4. Scrieti un program **http-echod.c** care modifica serverul **echod** scris in laboratorul anterior pentru a servi cereri TCP de la clienti pe portul 8080.

http-echod nu face decat sa trimita inapoi ca raspuns clientului propria lui cerere, dar in acest scop serverul trebuie sa formeze un raspuns corect HTTP, care presupune existenta unui header. Raspunsul serverului este un text care trebuie sa includa un header de tipul urmator:

```
"HTTP/1.1 200 OK\n"
"Content-Type: text/html; charset=utf-8\n\n"
```

Header-ul este despartit de restul raspunsului serverului HTTP printr-un dublu caracter newline.

http-echod copiaza cererea primita de la client, trimite header-ul de mai sus catre client urmat de copia cererii clientului si inchide conexiunea.

Folositi drept client de web orice browser (cel mai bine e sa testati cu mai multe browsere). Ca sa accesati serviciul tipariti in browser URL-ul urmator:

http://localhost:8080

Ce anume cititi in browser? Cum arata o cerere client de web? Cautati informatii pe net despre semnificatia campurilor pe care le identificati in cererile clientilor.

Odata ce aveti un server functional, transformati-l in demon folosind functia *daemon_init* scrisa la punctul 3.

Observatie: Dupa ce serverul se transforma in demon, **nu se pot folosi printf/scanf & co. pt a scrie/citi mesaje pe ecran/de la tastatura!** Nu se poate folosi decat functia *syslog* iar mesajele vor aparea in */var/log/syslog*.

5. Scrieti un program **httpd.c** care extinde **http-echod** a.i. dupa ce citeste cererea clientului sa identifice in cererea acestuia documentul web solicitat atunci cand clientul a accesat URL-ul *http://localhost:8080*.

Serverul trebuie sa formeze un raspuns corect HTTP, care presupune existenta unui header. La fel ca mai inainte, header-ul contine urmatoarele string-uri:

"HTTP/1.1 200 OK\n"
"Content-Type: text/html; charset=utf-8\n"

Atentie, de la ultima linie de mai sus a disparut un “\n”, pentru ca nu s-a terminat header-ul HTTP, serverul fiind obligat sa raspunda clientului furnizand continutul documentului solicitat.

Odata ce a identificat documentul (fisierul) solicitat de client, serverul trebuie sa completeze ultima linie a headerului cu dimensiunea fisierului, pe care o ataseaza in header astfel:

"Content-Length: <size>\n\n"

unde in loc de <size> veti trece dimensiunea fisierului cerut de client si returnata de apelul sistem *stat/fstat*.

Apoi **httpd** trimite header-ul astfel format catre client, mapeaza fisierul solicitat in memorie cu ajutorul *mmap* si trimite continutul zonei de memorie aferente documentului catre client, dupa care inchide conexiunea.

Observatie: daca intr-un URL nu apare o cerere explicita pentru un document web anume (i.e., un anumit nume de fisier html) ca in URL-ul de mai sus, presupunerea implicita a serverului este ca se solicita un document numit *index.html* (sau *index.php*, de pilda, daca vorbim de documente dinamice).

Indicatie: pentru simplitate, la inceput folositi un fisier *index.html* care redirecteaza clientul catre adresa *https://fmi.unibuc.ro*, ca mai jos:

```
<head>
  <meta http-equiv="Refresh" content="0; URL=https://fmi.unibuc.ro/" />
</head>
```

6. Instruiti clientul sa foloseasca un URL care refera un document HTML ceva mai sofisticat, de pilda fisierul *bosch.html* de mai jos:

```
<html>
<head>
<title>Example</title>
</head>
<body>
<h1>Ship of fools</h1>
<img src=\"bosch.jpg\" alt=\"ship of fools\" width=\"500\" height=\"600\">
</body>
</html>
```

Cum se schimba interactiunea clientului cu **httpd** atunci cand tipariti in browser URL-ul *http://localhost:8080/bosch.html*? De ce nu functioneaza serverul pe care l-ati scris la punctul anterior? Ce modificari trebuie sa faceti pentru ca **httpd** sa poate servi asemenea cereri?